🎬 Netflix Clone - Complete Project Explanation

📋 Table of Contents

1. Project Overview
2. Complete Tech Stack
3. Architecture & Design
4. Authentication Flow
5. Database Schema
6. API Architecture
7. Frontend Components
8. Security Implementation
9. UI/UX Design
10. Development Workflow

🎯 Project Overview

What is This Project?

A full-stack Netflix clone application that replicates the Netflix user interface and implements enterprise-grade security features including Multi-Factor Authentication (MFA) using authenticator apps.

Key Objectives

- **Modern Web Development**: Demonstrate proficiency in React.js and Express.js
- **Security-First Approach**: Implement TOTP-based MFA for enhanced security
- **Real-World Application**: Create a production-ready authentication system
- **Netflix UI Replication**: Build pixel-perfect Netflix-style interface
- **Full-Stack Integration**: Connect React frontend with Express/MongoDB backend

Use Cases

1. **Portfolio Project**: Showcase full-stack development skills
2. **Learning Resource**: Study modern authentication patterns
3. **Template**: Base for building secure web applications
4. **Netflix Clone**: Foundation for video streaming platforms

---

🛠️ Complete Tech Stack

Frontend Technologies

Core Framework

- **React 18.2.0**
  - Modern JavaScript library for building user interfaces
  - Component-based architecture

    - Virtual DOM for efficient rendering
    - Hooks for state management (useState, useEffect, useContext)

Routing

- **React Router DOM 6.20.0**
    - Client-side routing for single-page application
    - Protected route implementation
    - Programmatic navigation
    - URL parameter management

HTTP Client

- **Axios 1.6.2**
    - Promise-based HTTP client
    - Request/response interceptors
    - Automatic JSON transformation
    - Error handling middleware

UI Libraries

- **React Icons 4.12.0**
    - 40,000+ icons from popular icon libraries
    - Font Awesome, Material Design icons
    - Tree-shakable for smaller bundle size
- **React Toastify 9.1.3**
    - Toast notification system
    - Customizable appearance
    - Auto-dismiss functionality
    - Queue management

Build Tools

- **React Scripts 5.0.1**
    - Create React App configuration
    - Webpack bundling
    - Babel transpilation
    - Hot module replacement (HMR)

Backend Technologies

Server Framework

- **Express.js 4.18.2**
    - Fast, minimalist web framework for Node.js
    - Middleware architecture

    - RESTful API routing
    - Error handling

Database

- **MongoDB**
    - Document-oriented NoSQL database
    - Flexible schema design
    - BSON data format
    - Horizontal scalability
- **Mongoose 8.0.3**
    - MongoDB object modeling (ODM)
    - Schema validation
    - Query building
    - Middleware hooks
    - Data type casting

Authentication & Security

- **jsonwebtoken 9.0.2**
    - JWT creation and verification
    - Token payload encryption
    - Expiration management
    - Signature verification
- **bcryptjs 2.4.3**
    - Password hashing algorithm
    - Salt generation
    - Bcrypt encryption
    - Secure password comparison
- **speakeasy 2.0.0**
    - TOTP (Time-based OTP) generation
    - RFC 6238 compliant
    - Secret key management
    - Code verification
- **qrcode 1.5.3**
    - QR code generation for MFA setup
    - Multiple output formats (PNG, SVG, terminal)
    - Error correction levels
    - Custom styling options

Middleware & Utilities

- **cors 2.8.5**
  - Cross-Origin Resource Sharing
  - Allow frontend-backend communication
  - Configurable origin policies
- **dotenv 16.3.1**
  - Environment variable management
  - Secure configuration storage
  - Development/production separation
- **express-validator 7.0.1**
  - Request validation middleware
  - Sanitization functions
  - Custom validation rules

Development Tools

Process Management

- **nodemon 3.0.2**
  - Automatic server restart on file changes
  - Watch file patterns
  - Delay restart on rapid changes
- **concurrently 8.2.2**
  - Run multiple npm scripts simultaneously
  - Colored output for different processes
  - Kill all on first error

Code Quality

- **ESLint**
  - JavaScript linting
  - React plugin
  - Accessibility checks (jsx-a11y)
  - Code style enforcement
- **Babel**
  - JavaScript transpilation
  - ES6+ to ES5 conversion
  - JSX transformation
  - Plugin system

🏗 Architecture & Design

Application Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                              CLIENT SIDE                             │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────┐        │
│   │   Browser    │──▶│   React      │──▶│   Router     │        │
│   │ (Port 3000)  │     │ Components   │     │   (v6)       │        │
│   └──────────────┘     └──────────────┘     └──────────────┘        │
│          │                    │                    │                 │
│          │                    ▼                    │                 │
│          │             ┌──────────────┐            │                 │
│          │             │ Auth Context │            │                 │
│          │             │   (State)    │            │                 │
│          │             └──────────────┘            │                 │
│          │                    │                    │                 │
│          └────────────────────┼────────────────────┘                 │
│                               ▼                                      │
│                        ┌──────────────┐                              │
│                        │    Axios     │                              │
│                        │  API Client  │                              │
│                        └──────────────┘                              │
└──────────────────────────────┼───────────────────────────────────────┘
                               │
                        HTTP Requests
                               │
┌──────────────────────────────▼───────────────────────────────────────┐
│                              SERVER SIDE                             │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────┐        │
│   │   Express    │──▶│     CORS     │──▶│  Body Parser │        │
│   │ (Port 5000)  │     │  Middleware  │     │  Middleware  │        │
│   └──────────────┘     └──────────────┘     └──────────────┘        │
│          │                                          │                │
│          ▼                                          ▼                │
│   ┌──────────────┐                          ┌──────────────┐        │
│   │    Router    │                          │     Auth     │        │
│   │/api/auth/*   │──────────────────────────▶│  Middleware  │        │
│   └──────────────┘                          │(JWT Verify)  │        │
│          │                                  └──────────────┘        │
│          ▼                                                          │
│   ┌──────────────────────────────────────────────────────────┐      │
│   │                Authentication Routes                     │      │
│   │  • POST /signup       • POST /signin                     │      │
│   │  • POST /setup-mfa    • POST /verify-mfa-setup           │      │
│   │  • POST /verify-mfa   • GET /me                          │      │
│   └──────────────────────────────────────────────────────────┘      │
│                                  │                                  │
│                                  ▼                                  │
│                          ┌──────────────┐                           │
```

```
│                        │   Mongoose    │                        │
│                        │     ODM       │                        │
│                        └───────────────┘                        │
└─────────────────────────────────┼──────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────┐
│                       DATABASE LAYER                         │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │              MongoDB Database                        │    │
│  │         (mongodb://localhost:27017)                  │    │
│  │                                                      │    │
│  │  Database: netflix-clone                             │    │
│  │  Collection: users                                   │    │
│  │    - email (String, unique, required)                │    │
│  │    - password (String, hashed, required)             │    │
│  │    - mfaEnabled (Boolean, default: false)            │    │
│  │    - mfaSecret (String, encrypted)                   │    │
│  │    - createdAt (Date, auto)                          │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

Design Patterns Used

1. MVC Architecture (Modified)

- **Model**: MongoDB schemas (User.js)
- **View**: React components
- **Controller**: Express route handlers

2. Context API Pattern

- Centralized authentication state
- Provider-Consumer pattern
- Global state management without Redux

3. Middleware Pattern

- Request validation
- JWT verification
- Error handling
- CORS configuration

4. RESTful API Design

- Resource-based URLs
- HTTP verb usage (GET, POST, PUT, DELETE)
- Stateless communication

- JSON data format

5. Repository Pattern

- Mongoose models abstract database operations
- Separation of data access from business logic

🔐 Authentication Flow

1. User Registration Flow

```
┌──────────┐
│  User    │
└──────────┘
     │
     │
     │ 1. Enter email & password
     ▼
┌──────────────────┐
│   SignUp.js      │
│  (Frontend)      │
└──────────────────┘
     │
     │
     │ 2. POST /api/auth/signup
     │    { email, password }
     ▼
┌──────────────────┐
│  auth.js Route   │
│   (Backend)      │
└──────────────────┘
     │
     │
     │ 3. Validate input
     │ 4. Check if user exists
     ▼
┌──────────────────┐
│   bcryptjs       │
│  Hash password   │
└──────────────────┘
     │
     │
     │ 5. Save to database
     ▼
┌──────────────────┐
│    MongoDB       │
│  users: {...}    │
└──────────────────┘
     │
     │
     │ 6. Generate JWT token
     ▼
┌──────────────────┐
│  jsonwebtoken    │
│  Create token    │
└──────────────────┘
     │
```

```
      │
      │ 7. Return: { success, token, user }
      ▼
┌──────────────────┐
│   Frontend       │
│ Store in         │
│ localStorage     │
└──────────────────┘
```

2. MFA Setup Flow

```
┌──────────┐
│  User    │
└──────────┘
     │
     │
     │ 1. Click "Setup MFA"
     ▼
┌──────────────────┐
│   SignUp.js      │
│ (After signup)   │
└──────────────────┘
     │
     │
     │ 2. POST /api/auth/setup-mfa
     │    Headers: { Authorization: Bearer <token> }
     ▼
┌──────────────────┐
│  Auth Middleware │
│  Verify JWT      │
└──────────────────┘
     │
     │
     │ 3. JWT valid → Continue
     ▼
┌──────────────────┐
│   speakeasy      │
│ Generate secret  │
│ (32 char base32) │
└──────────────────┘
     │
     │
     │ 4. Create TOTP URL
     │    otpauth://totp/Netflix:user@email?secret=XXX
     ▼
┌──────────────────┐
│    qrcode        │
│  Generate QR     │
│  (Data URL)      │
└──────────────────┘
     │
     │
     │ 5. Return: { qrCode, secret }
     ▼
┌──────────────────┐
│   Frontend       │
```

```
  │  Display QR &    │
  │   Secret Key     │
  └──────────────────┘
       │
       │
       │ 6. User scans QR with authenticator app
       │    (Google/Microsoft Authenticator)
       ▼
  ┌──────────────────┐
  │ Authenticator    │
  │    App adds      │
  │    account       │
  └──────────────────┘
       │
       │
       │ 7. App generates 6-digit code
       │    (Changes every 30 seconds)
       ▼
  ┌──────────────────┐
  │     User         │
  │  Enters code     │
  └──────────────────┘
       │
       │
       │ 8. POST /api/auth/verify-mfa-setup
       │    { token: "123456" }
       ▼
  ┌──────────────────┐
  │   speakeasy      │
  │  Verify token    │
  │ (time-based)     │
  └──────────────────┘
       │
       │
       │ 9. Valid → Update user
       ▼
  ┌──────────────────┐
  │    MongoDB       │
  │ mfaEnabled: true │
  │ mfaSecret: XXX   │
  └──────────────────┘
```

3. Sign In with MFA Flow

```
  ┌──────────┐
  │  User    │
  └──────────┘
       │
       │
       │ 1. Enter email & password
       ▼
  ┌──────────────────┐
  │   SignIn.js      │
  │  (Frontend)      │
  └──────────────────┘
       │
       │
```

```
        │ 2. POST /api/auth/signin
        │    { email, password }
        ▼
    ┌──────────────────┐
    │  auth.js Route   │
    │   (Backend)      │
    └──────────────────┘
        │
        │
        │ 3. Find user in database
        ▼
    ┌──────────────────┐
    │    bcryptjs      │
    │ Compare password │
    └──────────────────┘
        │
        │
        │ 4. Password valid?
        ├─ No → Return error
        │
        │ 5. Check mfaEnabled
        ├─ No → Generate JWT, return token
        │
        │ 6. Yes → Generate temp token
        ▼
    ┌──────────────────┐
    │   Frontend       │
    │ Show MFA input   │
    │ mfaRequired:true │
    └──────────────────┘
        │
        │
        │ 7. User opens authenticator app
        │    Gets current 6-digit code
        ▼
    ┌──────────────────┐
    │     User         │
    │  Enters code     │
    └──────────────────┘
        │
        │
        │ 8. POST /api/auth/verify-mfa
        │    { tempToken, token: "123456" }
        ▼
    ┌──────────────────┐
    │   Verify temp    │
    │     token        │
    └──────────────────┘
        │
        │
        │ 9. Get user's mfaSecret
        ▼
    ┌──────────────────┐
    │   speakeasy      │
    │  Verify TOTP     │
    │  (30s window)    │
    └──────────────────┘
        │
        │
        │ 10. Valid → Generate full JWT
```

```
        ▼
    ┌──────────────────┐
    │   Frontend       │
    │ Store JWT token  │
    │ Navigate to /browse
    └──────────────────┘
```

4. Protected Route Access

```
    ┌──────────┐
    │  User    │
    └──────────┘
         │
         │
         │ 1. Navigate to /browse
         ▼
    ┌──────────────────┐
    │   React Router   │
    │  Check Auth      │
    └──────────────────┘
         │
         │
         │ 2. Read from AuthContext
         ▼
    ┌──────────────────┐
    │  AuthContext     │
    │ currentUser?     │
    └──────────────────┘
         │
         │
         ├─ No → Redirect to /signin
         │
         │ 3. Yes → API call with token
         ▼
    ┌──────────────────┐
    │  Axios Request   │
    │ Headers: Bearer  │
    └──────────────────┘
         │
         │
         │ 4. GET /api/auth/me
         ▼
    ┌──────────────────┐
    │  Auth Middleware │
    │  Extract token   │
    └──────────────────┘
         │
         │
         │ 5. jwt.verify(token, JWT_SECRET)
         ▼
    ┌──────────────────┐
    │  jsonwebtoken    │
    │  Decode payload  │
    └──────────────────┘
         │
         │
         ├─ Invalid → 401 Unauthorized
```

```
        │
        │ 6. Valid → Find user
        ▼
  ┌──────────────────┐
  │    MongoDB       │
  │  Return user     │
  └──────────────────┘
        │
        │
        │ 7. Send user data
        ▼
  ┌──────────────────┐
  │   Frontend       │
  │ Render Browse    │
  │     Page         │
  └──────────────────┘
```

💾 Database Schema

User Model (MongoDB/Mongoose)

```
{
  // Primary Identification
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true,
    match: /^[^\s@]+@[^\s@]+\.[^\s@]+$/  // Email regex
  },

  // Authentication
  password: {
    type: String,
    required: true,
    minlength: 6,
    // Stored as bcrypt hash: $2b$10$...
  },

  // Multi-Factor Authentication
  mfaEnabled: {
    type: Boolean,
    default: false
  },

  mfaSecret: {
    type: String,
    default: null,
    // Base32 encoded secret for TOTP
    // Example: "JBSWY3DPEHPK3PXP"
```

```
  },

  // Account Status
  emailVerified: {
    type: Boolean,
    default: true  // Simplified for this project
  },

  // Metadata
  createdAt: {
    type: Date,
    default: Date.now
  },

  updatedAt: {
    type: Date,
    default: Date.now
  }
}
```

Schema Hooks

```
// Pre-save hook for password hashing
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Instance method for password comparison
userSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};
```

Example Document

```
{
  "_id": "65a1b2c3d4e5f6a7b8c9d0e1",
  "email": "user@example.com",
  "password": "$2b$10$XGJ8KpR5nB7YxHZ4v9K1.eQY8Zx3WqR4nB7YxHZ4v9K1eQ",
  "mfaEnabled": true,
  "mfaSecret": "JBSWY3DPEHPK3PXPJBSWY3DPEHPK3PXP",
  "emailVerified": true,
  "createdAt": "2026-02-07T10:30:00.000Z",
  "updatedAt": "2026-02-07T10:45:00.000Z",
  "__v": 0
}
```

🌐 API Architecture

Authentication Endpoints

1. POST `/api/auth/signup`

Purpose: Register new user account

Request:

```
{
  "email": "user@example.com",
  "password": "securePassword123"
}
```

Process:

1. Validate email format and password length
2. Check if email already exists
3. Hash password with bcrypt (10 salt rounds)
4. Create user document in MongoDB
5. Generate JWT token (7-day expiry)
6. Return token and user data

Response (Success - 201):

```
{
  "success": true,
  "message": "User registered successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "65a1b2c3d4e5f6a7b8c9d0e1",
    "email": "user@example.com",
    "mfaEnabled": false
  }
}
```

Response (Error - 400):

```
{
  "success": false,
  "message": "Email already exists"
}
```

---

2. POST `/api/auth/signin`

Purpose: Authenticate existing user

Request:

```
{
  "email": "user@example.com",
  "password": "securePassword123"
}
```

Process:

1. Find user by email
2. Compare password using bcrypt
3. Check if MFA is enabled
4. If MFA disabled: Generate full JWT token
5. If MFA enabled: Generate temporary token (5-min expiry)

Response (No MFA - 200):

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "mfaRequired": false,
  "user": {
    "id": "65a1b2c3d4e5f6a7b8c9d0e1",
    "email": "user@example.com"
  }
}
```

Response (With MFA - 200):

```
{
  "success": true,
  "mfaRequired": true,
  "tempToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "message": "Please provide MFA code"
}
```

---

3. POST `/api/auth/setup-mfa`

Purpose: Generate QR code for MFA setup

Headers:

```
Authorization: Bearer <JWT_TOKEN>
```

Request: (No body required)

Process:

1. Verify JWT token in header
2. Generate random secret using speakeasy (32 chars, base32)
3. Create TOTP URL with app name and user email
4. Generate QR code as data URL using qrcode library
5. Return QR code and secret (for manual entry)

Response (200):

```
{
  "success": true,
  "qrCode": "...",
  "secret": "JBSWY3DPEHPK3PXPJBSWY3DPEHPK3PXP",
  "message": "Scan this QR code with your authenticator app"
}
```

---

4. POST `/api/auth/verify-mfa-setup`

Purpose: Verify MFA code and enable MFA for user

Headers:

```
Authorization: Bearer <JWT_TOKEN>
```

Request:

```
{
  "token": "123456",
  "secret": "JBSWY3DPEHPK3PXPJBSWY3DPEHPK3PXP"
}
```

Process:

1. Verify JWT token
2. Validate 6-digit TOTP code using speakeasy
3. Check code against secret (30-second time window)

4. If valid: Update user's mfaEnabled and mfaSecret
5. Generate new JWT token with updated user data

Response (Success - 200):

```
{
  "success": true,
  "message": "MFA enabled successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "65a1b2c3d4e5f6a7b8c9d0e1",
    "email": "user@example.com",
    "mfaEnabled": true
  }
}
```

Response (Error - 400):

```
{
  "success": false,
  "message": "Invalid MFA code"
}
```

---

5. POST `/api/auth/verify-mfa`

Purpose: Verify MFA code during sign-in

Request:

```
{
  "tempToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token": "123456"
}
```

Process:

1. Verify temporary token (expires in 5 minutes)
2. Extract user ID from temp token
3. Get user's mfaSecret from database
4. Verify TOTP code using speakeasy
5. If valid: Generate full JWT token (7-day expiry)

Response (Success - 200):

```
{
  "success": true,
  "message": "MFA verification successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "65a1b2c3d4e5f6a7b8c9d0e1",
    "email": "user@example.com",
    "mfaEnabled": true
  }
}
```

---

6. GET `/api/auth/me`

Purpose: Get current authenticated user

Headers:

```
Authorization: Bearer <JWT_TOKEN>
```

Process:

1. Verify JWT token
2. Extract user ID from token payload
3. Find user in database (exclude password)
4. Return user data

Response (200):

```
{
  "success": true,
  "user": {
    "id": "65a1b2c3d4e5f6a7b8c9d0e1",
    "email": "user@example.com",
    "mfaEnabled": true,
    "createdAt": "2026-02-07T10:30:00.000Z"
  }
}
```

---

7. POST `/api/auth/disable-mfa`

Purpose: Disable MFA for user account

Headers:

```
Authorization: Bearer <JWT_TOKEN>
```

Request:

```
{
  "password": "securePassword123"
}
```

Process:

1. Verify JWT token
2. Verify user's password
3. Update mfaEnabled to false
4. Clear mfaSecret
5. Return success message

Response (200):

```
{
  "success": true,
  "message": "MFA disabled successfully"
}
```

🎨 Frontend Components

Component Hierarchy

```
App.js (Root)
├── AuthProvider (Context)
│   └── Router
│       ├── Home.js (Route: /)
│       ├── SignUp.js (Route: /signup)
│       ├── SignIn.js (Route: /signin)
│       └── Browse.js (Protected Route: /browse)
```

1. **App.js** - Application Root

Purpose: Main application component with routing

Key Features:

- React Router v6 implementation
- Route configuration

- AuthProvider wrapper
- Toast notifications container

Routes:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/signup" element={<SignUp />} />
  <Route path="/signin" element={<SignIn />} />
  <Route path="/browse" element={<Browse />} />
</Routes>
```

---

2. **AuthContext.js** - Global State Management

Purpose: Manage authentication state across the app

State Variables:

```
{
  currentUser: {
    id: string,
    email: string,
    mfaEnabled: boolean
  } | null,

  loading: boolean
}
```

Methods:

- `initializeAuth()` - Load user from localStorage on mount
- `login(userData)` - Set current user
- `logout()` - Clear user and localStorage
- `updateUser(userData)` - Update current user data

Storage:

- localStorage key: `netflix_auth_token`
- Persists JWT token across sessions

---

3. **Home.js** - Landing Page

Purpose: Netflix-style landing page

Sections:

1. **Hero Section**

    - Large title and subtitle
    - Email input field
    - "Get Started" CTA button
    - Background gradient overlay

2. **Feature Cards** (3 sections)
    - Enjoy on your TV
    - Download and watch offline
    - Watch everywhere
    - Each with icon, title, and description

3. **FAQ Section**
    - Expandable accordion items
    - Common questions about Netflix
    - Smooth expand/collapse animation

4. **Footer**
    - Multiple link columns
    - Company information
    - Social links

Styling:

- Black background (#141414)
- Red accents (#e50914)
- Responsive grid layouts
- Hover effects and transitions

---

4. **SignUp.js** - Registration Component

Purpose: Multi-step user registration with MFA

Steps:

Step 1: Account Creation

```
State: {
  email: '',
  password: '',
  loading: false
}
```

- Email input (validation)
- Password input (min 6 chars)
- Submit button
- Link to sign in

Step 2: MFA Setup (Optional)

```
State: {
  qrCode: '...',
  secret: 'JBSWY3DP...',
  showSecret: false
}
```

- QR code display (280x280px)
- Manual secret key (toggle visibility)
- Instructions for scanning
- "Continue" and "Skip" buttons

Step 3: MFA Verification

```
State: {
  mfaCode: '',
  loading: false
}
```

- 6-digit code input
- Real-time validation
- Submit button
- Back button

Flow Logic:

```
handleSignUp() →
  handleSetupMFA() →
    handleVerifyMFA() →
      Navigate to /browse
```

---

5. **SignIn.js** - Login Component

Purpose: Authenticate users with optional MFA

Steps:

Step 1: Credentials

```
State: {
  email: '',
  password: '',
```

```
  loading: false
}
```

- Email and password inputs
- Remember me checkbox (optional)
- Sign in button

Step 2: MFA Verification (Conditional)

```
State: {
  mfaRequired: false,
  tempToken: '',
  mfaCode: '',
  loading: false
}
```

- Shown only if user has MFA enabled
- 6-digit code input
- Verify button
- Instructions text

Authentication Logic:

```
handleSignIn() {
  if (response.mfaRequired) {
    setStep(2);
    setTempToken(response.tempToken);
  } else {
    storeToken(response.token);
    navigate('/browse');
  }
}

handleVerifyMFA() {
  verifyCode(tempToken, mfaCode);
  storeToken(response.token);
  navigate('/browse');
}
```

---

6. **Browse.js** - Protected Content Page

Purpose: Main Netflix browsing interface

UI Components:

Header:

```
const [scrolled, setScrolled] = useState(false);

useEffect(() => {
  window.addEventListener('scroll', handleScroll);
}, []);
```

- Fixed navigation bar
- Netflix logo
- Navigation links (Home, TV Shows, Movies, etc.)
- User email display
- Logout button
- Dynamic background (transparent → solid on scroll)

Hero Banner:

- Full-width background image
- Featured content title
- Description text
- Action buttons (Play, More Info)
- Gradient fade-out effect

Content Rows (3 sections):

1. Popular on Netflix (7 items)
2. Trending Now (7 items)
3. New & Popular (6 items)

Movie Poster Card:

```
{
  id: number,
  title: string,
  image: 'https://images.unsplash.com/...'
}
```

Hover Effects:

- Scale up to 115%
- Show overlay with title
- Display play button
- Smooth transition (0.4s)

MFA Status Badge:

- Fixed bottom-right position
- Green border and text (#46d369)
- Lock icon
- "Protected with MFA" text

🔒 Security Implementation

1. Password Security

Hashing with bcrypt

```
// Salt rounds: 10 (2^10 = 1024 iterations)
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);

// Comparison
const isMatch = await bcrypt.compare(plainPassword, hashedPassword);
```

Why bcrypt?

- Slow by design (prevents brute force)
- Automatic salt generation
- Future-proof (configurable cost factor)
- Industry standard

---

2. JWT Token Security

Token Structure

```
Header.Payload.Signature
```

Header (Algorithm and type):

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload (Claims):

```
{
  "userId": "65a1b2c3d4e5f6a7b8c9d0e1",
  "email": "user@example.com",
  "iat": 1707305400,  // Issued at
  "exp": 1707910200   // Expires (7 days)
}
```

Signature:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  JWT_SECRET
)
```

Token Types

1. Full JWT (7-day expiry)

```
jwt.sign(
  { userId: user._id, email: user.email },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);
```

- Used after successful authentication
- Stored in localStorage
- Sent in Authorization header

2. Temporary Token (5-minute expiry)

```
jwt.sign(
  { userId: user._id, tempAuth: true },
  process.env.JWT_SECRET,
  { expiresIn: '5m' }
);
```

- Used for MFA flow
- Short lifespan for security
- Cannot access protected routes

---

3. TOTP-based MFA

How TOTP Works

Time-Based One-Time Password (RFC 6238):

```
TOTP = HOTP(Secret, Time)

Where:
- Secret: Shared key (32 characters, base32 encoded)
```

```
- Time: Current Unix timestamp / 30 seconds
- HOTP: HMAC-based OTP algorithm
```

Generation Process:

```
// Server (speakeasy)
const secret = speakeasy.generateSecret({
  length: 32,
  name: 'Netflix Clone',
  issuer: 'Netflix'
});

// Generate current code
const token = speakeasy.totp({
  secret: secret.base32,
  encoding: 'base32'
});
// Result: "123456"

// Verify code (30-second window ± 1 step = 90 seconds tolerance)
const verified = speakeasy.totp.verify({
  secret: secret.base32,
  encoding: 'base32',
  token: userEnteredCode,
  window: 1
});
```

Why TOTP is Secure:

- Time-synchronized
- No internet required
- Secret never transmitted
- 30-second rotation
- Resistant to replay attacks

---

4. API Security Middleware

Authentication Middleware

```
const verifyToken = (req, res, next) => {
  // 1. Extract token from header
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'No token provided'
```

```
      });
    }

    try {
      // 2. Verify token
      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      // 3. Attach user to request
      req.userId = decoded.userId;
      next();

    } catch (error) {
      return res.status(401).json({
        success: false,
        message: 'Invalid token'
      });
    }
  };
```

Protected Routes:

- `/api/auth/setup-mfa` → Requires valid JWT
- `/api/auth/verify-mfa-setup` → Requires valid JWT
- `/api/auth/me` → Requires valid JWT
- `/api/auth/disable-mfa` → Requires valid JWT

---

5. CORS Configuration

```
app.use(cors({
  origin: 'http://localhost:3000',  // Frontend URL
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

Why CORS?

- Prevents unauthorized cross-origin requests
- Allows frontend (port 3000) to access backend (port 5000)
- Protects against CSRF attacks

---

6. Environment Variables

Sensitive Data Protection:

```
JWT_SECRET=long_random_string_min_32_chars
MONGODB_URI=mongodb://localhost:27017/netflix-clone
```

```
SERVER_PORT=5000
```

Security Best Practices:

- Never commit `.env` to version control
- Use different secrets for dev/production
- Minimum 32-character JWT secret
- Rotate secrets periodically

---

7. Input Validation

Express Validator

```
const { body, validationResult } = require('express-validator');

// Signup validation
[
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 6 })
],
(req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  // Process request
}
```

Validation Rules:

- Email: Valid format, normalized
- Password: Minimum 6 characters
- MFA Code: 6 digits, numeric only

🎨 UI/UX Design

Netflix Design System

Color Palette

```
/* Primary Colors */
--netflix-black: #141414;      /* Background */
--netflix-red: #e50914;        /* Brand color */
--netflix-white: #ffffff;      /* Text */

/* Secondary Colors */
```

```
  --gray-900: #141414;           /* Dark background */
  --gray-800: #1f1f1f;           /* Card background */
  --gray-700: #2f2f2f;           /* Hover states */
  --gray-400: #808080;           /* Secondary text */

  /* Accent Colors */
  --green: #46d369;              /* Success (MFA badge) */
  --red: #e50914;                /* Error/Primary CTA */
```

Typography

```
  /* Font Family */
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
               'Roboto', 'Oxygen', 'Ubuntu', sans-serif;

  /* Font Sizes */
  --heading-xl: 3rem;            /* Hero titles */
  --heading-lg: 2rem;            /* Section headers */
  --heading-md: 1.5rem;          /* Card titles */
  --body-lg: 1.125rem;           /* Descriptions */
  --body-md: 1rem;               /* Default text */
  --body-sm: 0.875rem;           /* Small text */
```

Spacing System

```
  /* Consistent spacing scale */
  --space-xs: 0.5rem;    /* 8px */
  --space-sm: 1rem;      /* 16px */
  --space-md: 1.5rem;    /* 24px */
  --space-lg: 2rem;      /* 32px */
  --space-xl: 3rem;      /* 48px */
  --space-2xl: 4rem;     /* 64px */
```

UI Components

1. Horizontal Scrolling Carousel

CSS Implementation:

```
  .row-posters {
    display: flex;
    gap: 8px;
    overflow-x: scroll;
    overflow-y: hidden;
    padding: 20px 0;
```

```
  scrollbar-width: none;  /* Firefox */
  scroll-behavior: smooth;
}

.row-posters::-webkit-scrollbar {
  display: none;  /* Chrome, Safari */
}
```

JavaScript Scroll Enhancement (Optional):

```
// Smooth scroll on arrow click
const scrollLeft = () => {
  container.scrollBy({ left: -300, behavior: 'smooth' });
};

const scrollRight = () => {
  container.scrollBy({ left: 300, behavior: 'smooth' });
};
```

---

2. Poster Hover Animation

CSS Transform:

```
.poster {
  transition: all 0.4s cubic-bezier(0.4, 0, 0.2, 1);
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.3);
}

.poster:hover {
  transform: scale(1.15);
  z-index: 100;
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.8);
}

.poster:hover img {
  filter: brightness(0.6);
}

.poster-overlay {
  opacity: 0;
  transition: opacity 0.3s ease;
}

.poster:hover .poster-overlay {
  opacity: 1;
}
```

Result:

- Scales up 15% on hover
- Darkens image (60% brightness)
- Shows title and play button
- Elevates above other posters (z-index)

---

3. Dynamic Header

Scroll Detection:

```
const [scrolled, setScrolled] = useState(false);

useEffect(() => {
  const handleScroll = () => {
    setScrolled(window.scrollY > 50);
  };

  window.addEventListener('scroll', handleScroll);
  return () => window.removeEventListener('scroll', handleScroll);
}, []);
```

Conditional Styling:

```
.browse-header {
  background: linear-gradient(to bottom, rgba(0,0,0,0.7), transparent);
  transition: background 0.5s ease;
}

.browse-header.scrolled {
  background: #141414;  /* Solid black */
}
```

---

4. Form Inputs (Auth Pages)

Styled Input Fields:

```
.auth-input {
  width: 100%;
  padding: 16px 20px;
  background: #333;
  border: 1px solid #555;
  border-radius: 4px;
  color: #fff;
  font-size: 1rem;
```

```
  transition: border-color 0.3s;
}

.auth-input:focus {
  outline: none;
  border-color: #e50914;
  background: #454545;
}

.auth-input::placeholder {
  color: #8c8c8c;
}
```

Button Styles:

```
.auth-btn {
  width: 100%;
  padding: 16px;
  background: #e50914;
  border: none;
  border-radius: 4px;
  color: #fff;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.3s;
}

.auth-btn:hover {
  background: #f40612;
}

.auth-btn:disabled {
  background: #733;
  cursor: not-allowed;
}
```

---

5. Toast Notifications

React Toastify Configuration:

```
toast.success('Logged in successfully', {
  position: 'top-right',
  autoClose: 3000,
  hideProgressBar: false,
  closeOnClick: true,
  pauseOnHover: true,
  draggable: true
```

```
});

toast.error('Invalid credentials', {
  position: 'top-right',
  autoClose: 5000
});
```

Custom Styling:

```
.Toastify__toast--success {
  background: #46d369;
}

.Toastify__toast--error {
  background: #e50914;
}
```

Responsive Design

Breakpoints

```
/* Mobile */
@media (max-width: 550px) {
  .hero-title { font-size: 1.5rem; }
  .poster { min-width: 200px; height: 112px; }
}

/* Tablet */
@media (max-width: 950px) {
  .header-nav { display: none; }  /* Hide nav on mobile */
  .poster { min-width: 250px; height: 140px; }
}

/* Desktop */
@media (min-width: 1400px) {
  .poster { min-width: 350px; height: 197px; }
}
```

Mobile Optimization

- Stack elements vertically
- Larger touch targets (min 44x44px)
- Simplified navigation
- Reduced poster sizes
- Hidden secondary elements

⚙️ Development Workflow

Setup Process

1. **Clone Repository**

```
git clone https://github.com/S-Rahul-Naik/Netflix-MFA.git
cd Netflix-MFA
```

2. **Install Dependencies**

```
npm install
```

3. **Configure Environment**

```
copy .env.example .env
# Edit .env with your settings
```

4. **Start MongoDB**

```
# Local MongoDB
mongod

# Or use MongoDB Atlas connection string
```

5. **Run Development Servers**

```
npm run dev
# Starts both backend (5000) and frontend (3000)
```

Development Scripts

```
{
  "scripts": {
    // Run full application
    "start": "concurrently \"npm run server\" \"npm run client\"",
    "dev": "concurrently \"npm run server\" \"npm run client\"",
    "dev:full": "concurrently \"npm run server\" \"npm run client\"",

    // Individual servers
```

```
    "server": "nodemon server/server.js",
    "client": "react-scripts start",

    // Production
    "build": "react-scripts build",

    // Testing
    "test": "react-scripts test"
  }
}
```

Git Workflow

```
# Initialize repository
git init

# Add files
git add .

# Commit changes
git commit -m "Descriptive message"

# Add remote
git remote add origin <repository-url>

# Push to GitHub
git push -u origin main
```

Debugging Tips

1. Backend Debugging

```
// Add logging
console.log('📥 Request:', req.body);
console.log('✅ Response:', response);

// MongoDB queries
User.find().then(users => console.log('Users:', users));

// JWT verification
try {
  const decoded = jwt.verify(token, JWT_SECRET);
  console.log('Decoded:', decoded);
} catch (err) {
  console.error('JWT Error:', err.message);
}
```

---

2. Frontend Debugging

```
// React DevTools
console.log('State:', state);
console.log('Props:', props);

// API calls
axios.post('/api/auth/signin', data)
  .then(res => console.log('Success:', res.data))
  .catch(err => console.error('Error:', err.response.data));

// localStorage
console.log('Token:', localStorage.getItem('netflix_auth_token'));
```

3. Network Tab

- Check request headers (Authorization)
- Verify request payload
- Inspect response status and data
- Look for CORS errors

Testing Checklist

Authentication Flow

- ☐ Sign up with new email
- ☐ Sign up with existing email (should fail)
- ☐ Sign in with correct credentials
- ☐ Sign in with wrong password (should fail)
- ☐ Access protected route without login (redirect to signin)

MFA Flow

- ☐ Setup MFA - generate QR code
- ☐ Scan QR with authenticator app
- ☐ Verify code from app
- ☐ Sign out and sign in with MFA code
- ☐ Try invalid MFA code (should fail)
- ☐ Try expired MFA code (should fail)

UI/UX

- ☐ Responsive design on mobile
- ☐ Horizontal scrolling works
- ☐ Hover effects on posters
- ☐ Header changes on scroll

- ☐ Toast notifications appear
- ☐ Loading states display correctly

Performance Optimization

1. Code Splitting

```
// Lazy load routes
const Browse = lazy(() => import('./pages/Browse'));
const SignUp = lazy(() => import('./pages/SignUp'));

<Suspense fallback={<Loading />}>
  <Routes>
    <Route path="/browse" element={<Browse />} />
  </Routes>
</Suspense>
```

2. Image Optimization

- Use appropriate image sizes (400x225 for posters)
- Lazy loading for off-screen images
- WebP format with fallbacks
- CDN hosting (Unsplash)

3. Bundle Optimization

```
# Analyze bundle size
npm run build
npx source-map-explorer 'build/static/js/*.js'
```

4. Database Indexing

```
// Create index on email field
userSchema.index({ email: 1 });
```

Deployment Guide

Frontend (Vercel/Netlify)

1. Build production bundle: `npm run build`
2. Deploy `build` folder
3. Set environment variables

4. Configure API endpoint to production backend

Backend (Heroku/Railway)

1. Set environment variables (JWT_SECRET, MONGODB_URI)
2. Ensure MongoDB connection string is correct
3. Deploy server folder
4. Test API endpoints

Environment Variables

```
# Production
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/netflix
JWT_SECRET=long_random_production_secret_minimum_32_chars
SERVER_PORT=5000
NODE_ENV=production
```

📚 Learning Outcomes

Skills Demonstrated

1. **Full-Stack Development**
   - React frontend with Express backend
   - RESTful API design
   - Database integration
2. **Authentication & Security**
   - JWT token implementation
   - Password hashing with bcrypt
   - TOTP-based MFA
   - Protected routes
3. **Modern JavaScript**
   - ES6+ features (async/await, destructuring, arrow functions)
   - React Hooks (useState, useEffect, useContext)
   - Promise handling
4. **Database Design**
   - MongoDB schema design
   - Mongoose ODM
   - Data validation
5. **UI/UX Design**
   - Responsive layouts

    - CSS animations
    - Hover effects
    - User-friendly forms

6. **DevOps Basics**
    - Environment configuration
    - Git version control
    - Deployment process

🎓 Conclusion

This Netflix clone demonstrates a production-ready authentication system with industry-standard security practices. The project showcases modern web development techniques, including:

- Secure user authentication with JWT
- Time-based one-time password (TOTP) MFA
- MongoDB database integration
- Netflix-inspired user interface
- Responsive design for all devices

The application serves as both a learning resource and a portfolio piece, illustrating proficiency in full-stack development, security implementation, and user experience design.

Project By: [Your Name]
Repository: https://github.com/S-Rahul-Naik/Netflix-MFA
Date: February 2026
License: MIT

📞 Support & Contact

For questions or issues:

- Open an issue on GitHub
- Check README.md for setup instructions
- Review SETUP_GUIDE.md for troubleshooting

Happy Coding! 🚀